Design and Analysis of Algorithms 2016
National University of Singapore CS3230
Prof Lee Wee Sun

Homework 3 (Due Date: Friday 28 October)

Please write the following on your homework:

- Name
- Collaborators (write none if no collaborators)
- Source, if you obtained the solution through research, e.g. through the web.

While you may collaborate, you *must write up the solution yourself* and you are asked to try doing the questions yourself first before discussing with others.

If you are asked to describe an algorithm, you should first provide a short description of the problem and what your main results are. Then you should provide a description of the algorithm in English, and if helpful, in pseudo-code. An example, or diagram can often be helpful in helping describe the algorithm. You should provide a proof (or indication) of correctness of the algorithm, and an analysis of the running time of the algorithm. Remember that you need to communicate clearly. Grades cannot be awarded if the grader is unable to understand what you are writing.

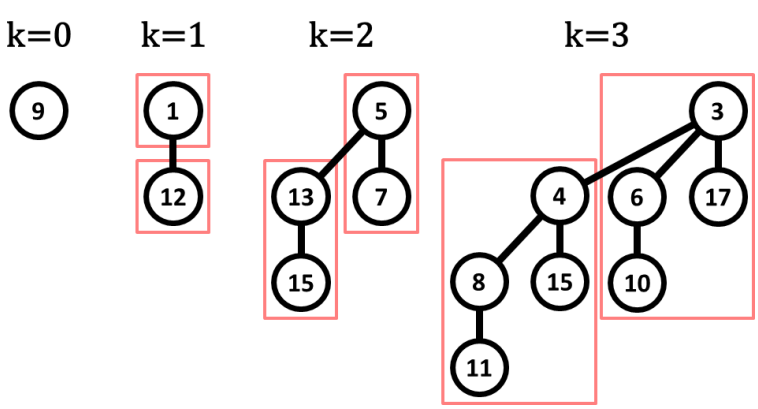
---

1. **Binomial Heap Insertion**
   A priority queue implemented using a min-heap has DELETE-MIN and INSERT operations that take $\Theta(\log n)$ time respectively. In this question, we will look at Binomial Heap, which can implement a priority queue with $\Theta(\log n)$ DELETE-MIN and amortised $\Theta(1)$ INSERT operations. A sequence of n insert operations take $\Theta(n \log n)$ time using a min-heap, but only $\Theta(n)$ time using a Binomial Heap.

   A Binomial Heap is implemented as a collection of Binomial Trees.
   A Binomial Tree is defined recursively as follows:

   - A binomial tree of order 0 is a single node.
   - A binomial tree of order k is formed from merging two binomial trees of order $k-1$.

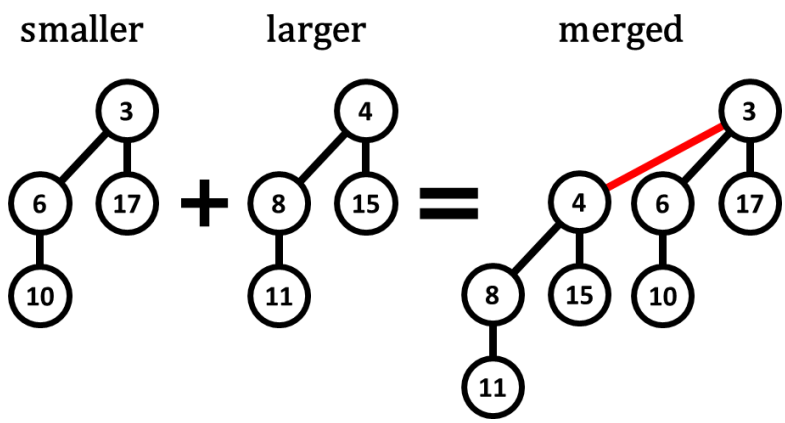
   Thus a binomial tree of order k has 2^k elements. The binomial trees for orders 0 to 3 are shown below.



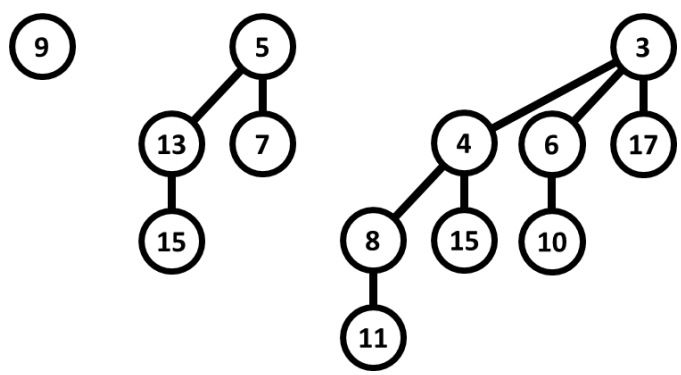
Similar to Binary Heaps, Binomial Trees have the additional property that each element in the tree is smaller than each of its children. We note that two Binomial Trees of the same size can be merged in $O(1)$.

To merge two Binomial Trees of the same size, we compare the roots of the two trees. The tree with the larger root is made a child of the root of the other tree.

An illustration is as follows:



Finally, a Binomial Heap (not a Binomial Tree!) is made out of a collection of Binomial Trees, all having different sizes.
For example, here is a Binomial Heap with 13 elements.
It consists of the trees of orders 0, 2 and 3.



(a) For a Binomial Heap containing 27 elements, what are the orders of the Binomial Trees that make up the Binomial Heap?

(b) For a Binomial Heap with n elements, give (and prove) an upper bound for the number of trees in the Binomial Heap. Also show that this upper bound is tight (i.e. it can be attained). Use exact rather than asymptotic notation for this question.

This also shows that the minimum value of the Binomial Heap can be found in $O(\log n)$. (As the minimum-valued node is a root of one of the Binomial Trees)

INSERT
Inserting a new element v into the Binomial Heap is done as follows:

- First create a new tree of order 0 (containing a single node v)
- While there exists two trees of the same order, merge them into the tree of the next larger order.

This ensures that there is only one tree of each order after each insertion.
For each $k \in \{0, 1, 2, \cdots\}$, we define T_k to be the number of trees of order k. Each T_k is thus always either 0 or 1.

(c) Give the values of $(T_0, T_1, T_2, T_3, T_4)$ for a Binomial Heap with 27 elements.

(d) Give the new values of $(T_0, T_1, T_2, T_3, T_4)$ for the Binomial Heap in (c) after adding one new element.

(e) Use the Accounting or Potential method to prove that the INSERT operation runs in amortised $O(1)$.

Recommended Strategy:

- First define an invariant for how much "money" or "potential" is stored in the data structure based on the values of T_k.
- Then prove that the invariant holds after every operation, using a constant cost for each operation.

2. **Arbitrage**

Arbitrage takes advantage of the discrepencies in currency exchange rate to make a profit through a sequence of trades. For example, if

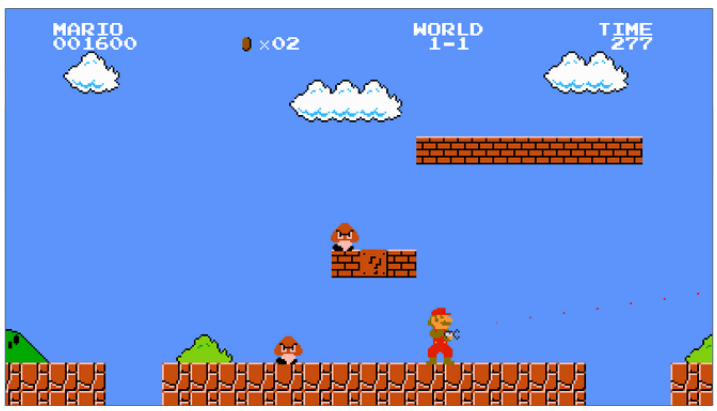
- 1 Singapore dollar buys 0.97 Australian dollar
- 1 Australian dollar buys 0.76 US dollars
- 1 US dollar buys 1.42 Singapore dollars,

then a smart trader who uses 1 Singapore dollar to buy Australian dollar, then converting it to US dollar and finally converting it back to Singapore dollar would get back around 1.047 dollars, earning 4.7% profit. Assume that the exchange rate is constant for a time period that is long enough to make k trades. Let $R[i, j]$ be the exchange rate table which denotes how much of currency j can be purchased using one dollar of currency i. Give a dynamic

programming algorithm for finding the maximum amount that can be made by starting with one dollar of currency 1 and doing k trades, converting back to currency 1 on the k-th trade. (As usual, when describing an algorithm, you should give some indication of correctness and analysis of the running time.)

3. **Would you like to build some bridges?**

   Mario is on a mission to save Princess Peach! As in the classical *Mario* games, our protagonist has to navigate through the world by jumping from one platform to another until he reaches Bowser's castle, where Princess Peach is held captive. However, Mario realised that jumping across platforms is a dangerous affair: He will perish if he were to drop in between platforms! As such, he has decided to build bridges in between platforms so that he can reach his destination safely.



   However, Mario might not have enough resource to build bridges in between all platforms. In addition, Mario is not very smart, and he cannot figure out if he has enough bridges to cover all gaps.

   To be specific, Mario's world is a one-dimensional world with n gaps which must be covered by bridges. At both ends of each gap, there are bridge stands where a bridge can be placed on. The gaps and corresponding bridge stands are of different lengths for each gap. Gap i is specified by two numbers (g_i, t_i) where g_i is the gap length and t_i is the total length of the gap plus the stands on both sides. Mario is given n bridges whereby the j-th bridge has length a_j. Bridge j can be used to cover gap i **if and only if** $g_i \leq a_j \leq t_i$.

   (a) Suggest a greedy algorithm that can be used to solve Mario's problem.

   Hint: One way is to sort the bridges first, then find a gap that is appropriate for the bridges from the shortest to the longest bridge.

   (b) Prove the correctness of your algorithm.

   Hint: First, show that the algorithm does not return a false positive (this should be straightforward). The key is to show that the algorithm does not return a false negative. One way to do that is to first argue that optimal substructure exists which allows us to

to select one bridge-gap pair then optimally solve for the remaining subproblem. Then argue that greedy choice holds for the problem, i.e. there exists a solution which selects the same first bridge-gap pair as your algorithm.

(c) Choose appropriate data structures to implement the above algorithm and suggests the runtime of your solution. You are encouraged to describe the steps in the execution of your algorithm using pseudocode or other means.